Predicting a Heavy-Tailed Income Density

Data Science for Business I – Assignment

Nooa von Schöneman

1. Introduction

This assignment aims to generate a dataset loosely resembling income distribution where the vast majority of observations fall under a bell curve, but a small subset of high earners can reach values magnitudes larger than the rest. I analyze how much data is required to predict the tail of such function and what it takes to break the models.

2. Simulating the data

The simulated data is a density-function that creates an area of one by weighted sum of its two parts. The body is a lognormal with a log-mean of 10.5 and log-standard-deviation of 0.5, and the tail consists of a Lomax (Pareto type II) distribution that falls far slower than the lognormal. ε assigns the probability of a random observation falling for either the body or the tail. This means that for small values of ε, the vast majority of observations fall under the bell curve that's resistant to outliers, while the minor share of the sample that's assigned to the tail has much more variation in its income levels. Tail observations are defined as incomes above 250,000 €.

I use a ε of 0.025 as a baseline, which gives a random observation a 3.60% chance of earning above 100,000€, a 0.88% chance of earning above 250,000€ and a 0.22% chance of earning over a million euros. Attached below is the full equation, and a graph of the function in normal and log units along with an example dataset (Figure 1).

$$f(x) = (1-\varepsilon)\frac{1}{x \cdot 0.5\sqrt{2\pi}}\exp\left(-\frac{(\ln x - 10.5)^2}{2 \cdot 0.5^2}\right) + \varepsilon\frac{1.5}{250000}\left(1+\frac{x}{250000}\right)^{-2.5}, \quad x > 0$$

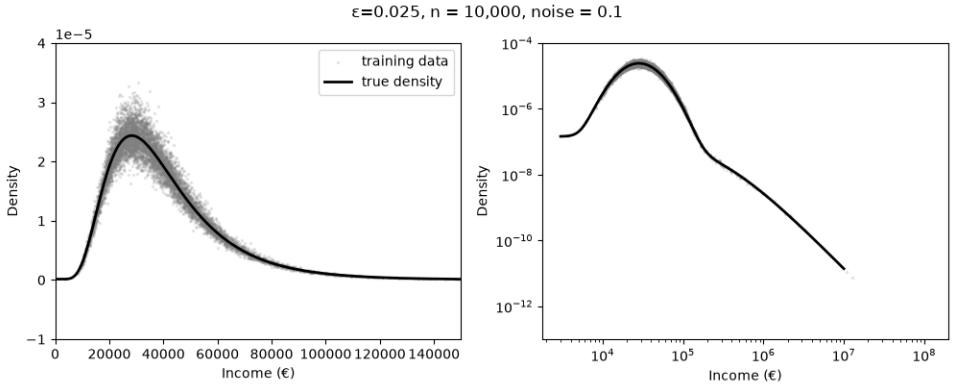


Figure 1. True density and an example dataset (n = 10,000, ε = 0.025, noise = 0.1).

Each observation has an income, the true density f(income) and a noisy density f(income)·e^(s·Z), where Z is standard normal and s is the noise level (s = 0.1 is roughly a 10% error). The models are trained on the noisy density to predict it from income, and are scored against the true density, which they never see.

3. Fitting the models

Since most of the density lies in the body, predictive models should be able to get good predictions for that part much sooner than the tail. As such, the question becomes just how much data is required to also get good predictions for the tail and how parameters like ε and noise affect the models. Since the nature of the data is nonlinear, it follows that the models too should be able to deal with nonlinearities. I trained three different models on income and the noisy density.

1. **Spline OLS:** a smooth curve glued together from cubic pieces at 8 points, fitted in logs and continued as a straight line beyond the data.
2. **K-nearest neighbors:** the prediction is the average of the 15 training people with the closest incomes.
3. **Gradient boosting:** many small decision trees, each correcting the errors of the previous ones.

The base models were trained with 10 unique datasets of 10,000 observations, a ε of 0.025 and noise levels of 0.1 or 10%. 75% of the data was used for training and 25% for testing. Scoring compares each prediction with the true density and uses the median of the 10 runs. R^2 and mean squared error are computed on the test set, but since almost everyone belongs to the body, they don't say much about how well the models predict the tail. As such, a "factor" score is also calculated where 1.0 means a perfect prediction, 2.0 means that the typical prediction is off by a factor of two and so forth. The body factor consists of incomes from 10,000 to 150,000 €, and the tail is split into 250,000 to 1M € and 1M to 10M € income brackets. The results are in Table 1 and the fitted curves in Figure 2.

Model	R^2	Body factor	Tail 250k–1M factor	Tail 1M–10M factor
Spline OLS	0.991	1.08	1.37	1.40
k-NN	0.996	1.02	1.05	3.51
Gradient boosting	0.998	1.04	1.90	6.31

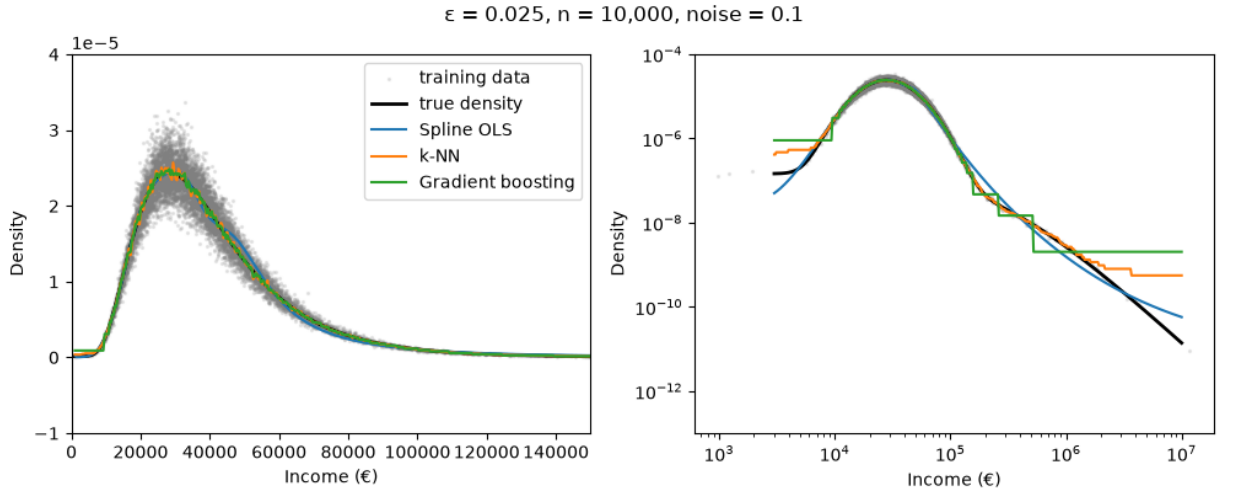


Figure 2. Fitted curves against the true density (n = 10,000, ε = 0.025, noise = 0.1).

All three nonlinear models perform well for the body, and I would say that Spline OLS and k-NN manage predictions quite well for the tail as well despite there only being an average of 88 tail observations in the dataset. Gradient boosting begins to give the weakest results once we reach very high incomes, though it reaches the highest overall R^2.

4. Changing the parameters

To see more clearly what it takes to break the tail, I re-ran the regressions with different levels of noise, ε and observation counts. Like above, for every combination of data size, noise and ε, 10 unique datasets were generated from which the median score is reported.

4.1 Data amount & tail weight

The tail can only be learned when the data contains enough tail observations, which can be increased with more data or a heavier tail: for 10,000 observations about 35, 88 and 177 land above 250,000 € for ε = 0.01, 0.025 and 0.05. Table 2 shows the far-tail median error factor for different data amounts (noise = 0.1, ε = 0.025) and Table 3 for different ε (n = 10,000, noise = 0.1).

Far tail (1M–10M)	n = 1,000	n = 10,000	n = 100,000	n = 1,000,000
Spline OLS	2.64	1.40	1.39	1.38
k-NN	103.64	3.51	1.14	1.02
Gradient boosting	231.92	6.31	7.06	7.12

With this dataset, spline OLS seems to be on average surprisingly resistant to data-loss and with just 1,000 observations manages to outperform k-NN trained on 10,000 observations. k-NN improves the most from additional data and manages almost perfect predictions by n = 1,000,000. More data doesn't automatically mean a better model though and the gains for Spline OLS and Gradient boosting plateau after n = 10,000 (boosting's change from 6.31 to 7.12 is within run-to-run variation). K-NN and trees predict a constant beyond the data, while the spline continues in a straight line that is close to the true tail but not exact.

Far tail (1M–10M)	ε = 0.01	ε = 0.025	ε = 0.05
Spline OLS	3.0	1.40	1.2
k-NN	12.9	3.51	2.1
Gradient boosting	33.7	6.31	4.1

A heavier tail works like more data, since it increases the number of tail observations. From ε = 0.01 to 0.05 the far-tail error of the spline falls from 3.0 to 1.2 and of k-NN from 12.9 to 2.1. A larger ε is however a different population, not the same problem made easier. Boosting stays at least 4 times off at every ε.

4.2 Noise level

The noise is multiplicative, so 0.1 means roughly a 10% error, 0.5 a 50% error and so forth. Below are the R^2 scores on a dataset with the base specifications and different noise levels.

R^2 (n=10,000, ε=0.025)	Noise = 0.1	Noise = 0.3	Noise = 0.5	Noise = 0.8	Noise = 1
Spline OLS	0.991	0.991	0.992	0.992	0.985
k-NN	0.996	0.967	0.910	0.771	0.587
Gradient boosting	0.998	0.984	0.955	0.897	0.829

Overall, all models seem to be quite resistant to noise when compared to data quantity. Since noise is random, it doesn't bias the estimators on average but rather adds variance to them. Spline OLS seems almost immune, while k-NN (R^2 of about 0.6 at noise 1.0, even with 1,000,000 observations) and Gradient boosting become clearly worse from noise levels of 0.3–0.5 onward.

5. Conclusions

The body is much easier to predict while the tail requires far more data. However, observations only help if a model can use it. Spline OLS was able to get surprisingly decent results even with limited datasets such as n = 10,000, ε = 0.025, but couldn't improve meaningfully as the dataset grew. On the other hand, k-NN performed much worse with the tail on lower observation counts but kept improving steadily even when large datasets were further increased.

If there's one takeaway from this assignment, it's that the optimal model depends on the dataset at hand. In this case, spline performed the best when the dataset was more limited or extremely noisy, while k-NN was better with larger datasets that didn't contain much noise as illustrated by the two Figures below.

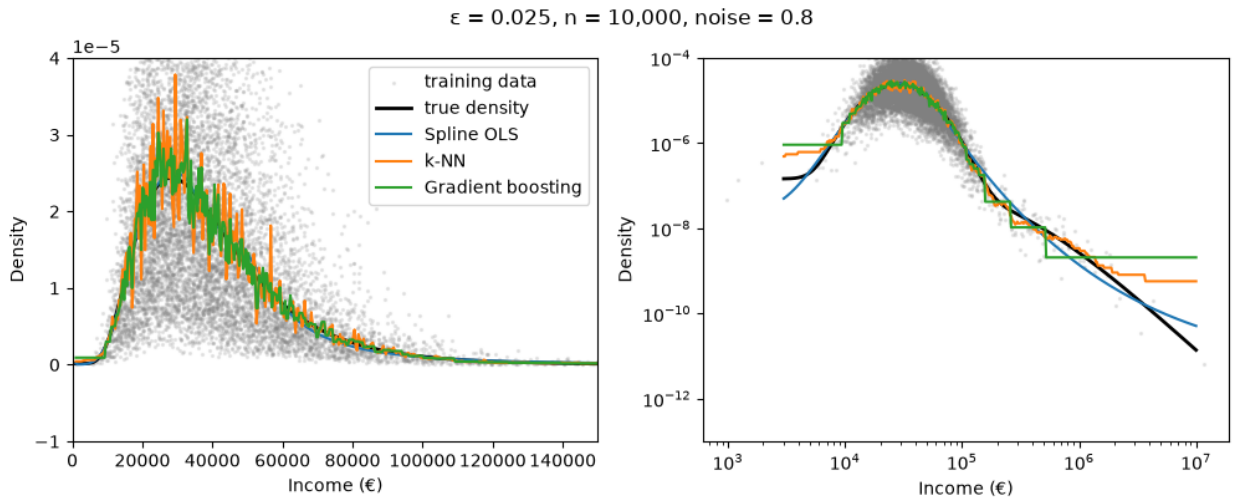


Figure 3. Very noisy data (n = 10,000, noise = 0.8): Spline OLS was the best choice for noisy / limited data.

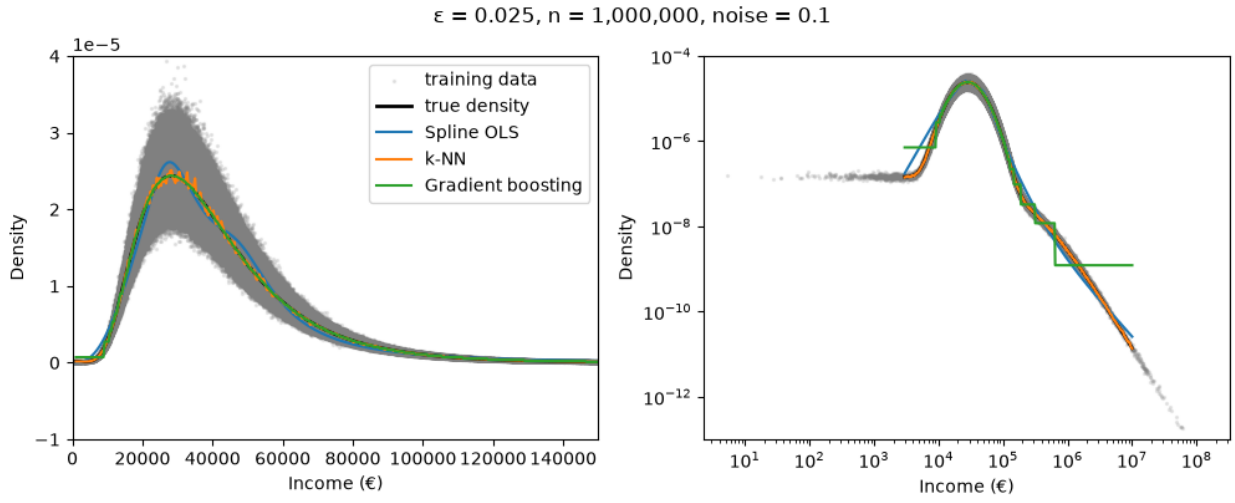


Figure 4. A large clean dataset (n = 1,000,000, noise = 0.1): k-NN performed the best with large, clean datasets.